

计算机图形学

实验四：光子映射算法

光子映射

- 和光线追踪一样，是真实感渲染领域的一种渲染方法。
- 能够高效地解决光线追踪类方法相对难以处理的焦散、间接光照等复杂现象。
- 以一定的内存空间为代价，获得更高的焦散渲染效率。

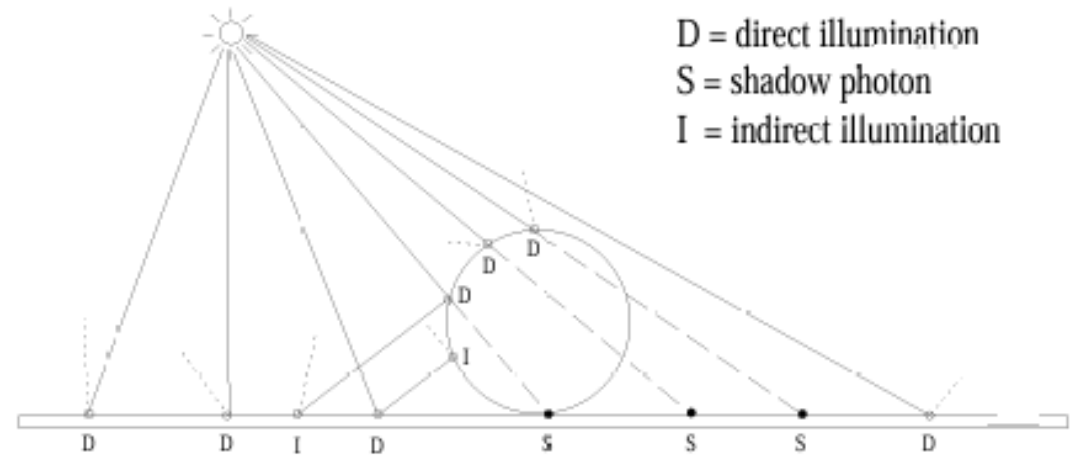


算法介绍-光子映射 (Photon mapping)

- 由Henrik Wann Jensen提出。
- 是一种双向的全局光照算法。通过密度估计来近似求解渲染方程中的积分项，用离散的光子样本估计连续的光照分布。
- 将渲染过程分为两个独立的阶段：
 - 构建光子图：从光源出发追踪"光子"，在漫反射表面存储光照信息；
 - 渲染：从相机出发进行渲染，查询存储的光子信息计算间接光照。

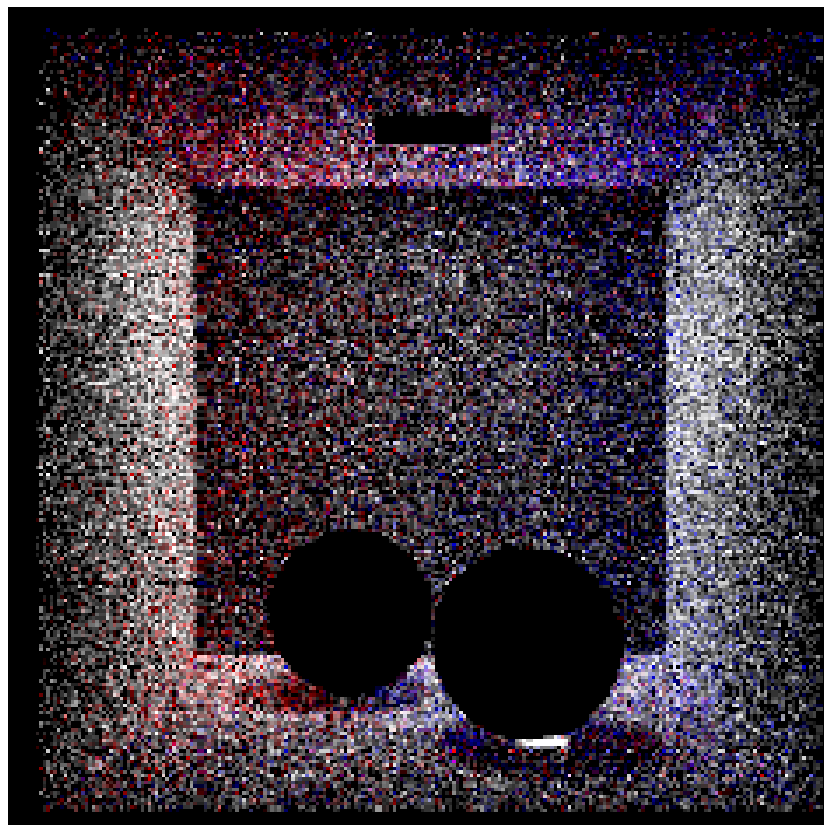
算法介绍-光子映射 (Photon mapping)

- 光子图的构建通过从场景中的光源发射大量光子实现。
- 每个光子在场景中的追踪方式与路径追踪类似。
- 当光子击中一个表面时，它会被存储在光子图中，并利用俄罗斯轮盘赌方法来决定该光子是被吸收还是被反射。
- 反射光子的新方向根据表面的BRDF计算。



算法介绍-光子映射 (Photon mapping)

- 全局光子图：
 - 存储从光源发射的所有光子的通量信息。
 - 从光源发射光子，并让光子在场景中反复弹射，每次击中漫反射表面都进行一次光子的记录，直到被某个漫反射表面彻底吸收掉为止。



算法介绍-光子映射 (Photon mapping)

- 全局光子图的构建过程：
 - 基于光源采样，确定射光线方向和初始光线的功率
 - 光线在场景中迭代，找到交点
 - 生成交点处的反射分布BSDF
 - 基于该BSDF计算反射分布，并记录反射方向和反射类型
 - 如果反射类型是漫反射，就记录到光子图
 - 使用俄罗斯轮盘赌判断光子是否被吸收
 - 如果没有被吸收，则更新下一次迭代的功率值
 - 更新下一次迭代的新的光线方向

算法介绍-光子映射 (Photon mapping)

- 光子图存储在一个平衡的KD树中。
- 平衡的KD树：
 - 既紧凑又高效
 - 平衡树保证了在具有N个光子的树中定位M个光子的时间复杂度为 $O(M \log_2(N))$ 。
 - 提高了渲染效率
 - 减少了每个光子命中所需的内存占用。

算法介绍-光子映射 (Photon mapping)

- 最终图像采用蒙特卡洛光线追踪技术进行渲染，其中像素辐射亮度的计算通过平均多组采样估计值实现。
- 每次采样需完成以下流程：
 - 从人眼视角发出一条射线，使其穿过像素平面射入场景空间。当射线与首个表面相交时，计算该射线返回的辐射亮度值。

算法介绍-光子映射 (Photon mapping)

- 表面辐射亮度 $L_s(x, \Psi_r)$ 的渲染公式:

$$L_s(x, \Psi_r) = L_e(x, \Psi_r) + \int_{\Omega} f_r(x, \Psi_i; \Psi_r) L_i(x, \Psi_i) \cos \theta_i d\omega_i$$

- 参数介绍:
 - L_s : 从交点 x 沿光线方向 Ψ_r 离开表面的光能
 - L_e : 表面自身发出的亮度
 - L_i : Ψ_i 方向上的亮度
 - f_r : 双向反射分布函数 (BRDF)
 - Ω : 入射方向构成的半球空间

算法介绍-光子映射 (Photon mapping)

- 表面辐射亮度 $L_s(x, \Psi_r)$ 的渲染公式:

$$L_s(x, \Psi_r) = L_e(x, \Psi_r) + \int_{\Omega} f_r(x, \Psi_i; \Psi_r) L_i(x, \Psi_i) \cos \theta_i d\omega_i$$

- 积分值 $L_r = \int_{\Omega} f_r(x, \Psi_i; \Psi_r) L_i(x, \Psi_i) \cos \theta_i d\omega_i$ 取决于场景中其他位置的辐射亮度值, 可采用路径追踪等蒙特卡洛方法直接求解。然而这种方法计算成本极高, 若结合光子映射技术, 即可获得更高效的求解方案。

算法介绍-光子映射 (Photon mapping)

- 对积分值 L_r 进行计算:
$$L_r(x, \Psi_r) = \int_{\Omega} f_r(x, \Psi_r, \Psi_i) \frac{d^2\Phi_i(x, \Psi_i)}{dA d\omega_i} d\omega_i$$

$$\approx \sum_{p=1}^N f_r(x, \Psi_r, \Psi_{i,p}) \frac{\Delta\Phi_p(x, \Psi_{i,p})}{\pi r^2}$$

- 公式含义: 先求附近区域内的所有通量, 然后再除以这个估算区域的表面积
- 计算表面交点x处的亮度 L_r , 通过以下步骤实现:
 - 定位距离x最近的N个光子: 以交点x为中心构建球体, 使其持续膨胀直至恰好包含N个光子, 此时球体半径记为r (思考: 这与取固定的r效果有何区别)
 - 基于每个光子p代表从 $\Psi_{i,p}$ 方向抵达x的光通量 Φ_p 的假设, 将光子信息代入渲染方程进行积分计算

算法介绍-光子映射 (Photon mapping)

- 对积分值 L_r 进行计算:

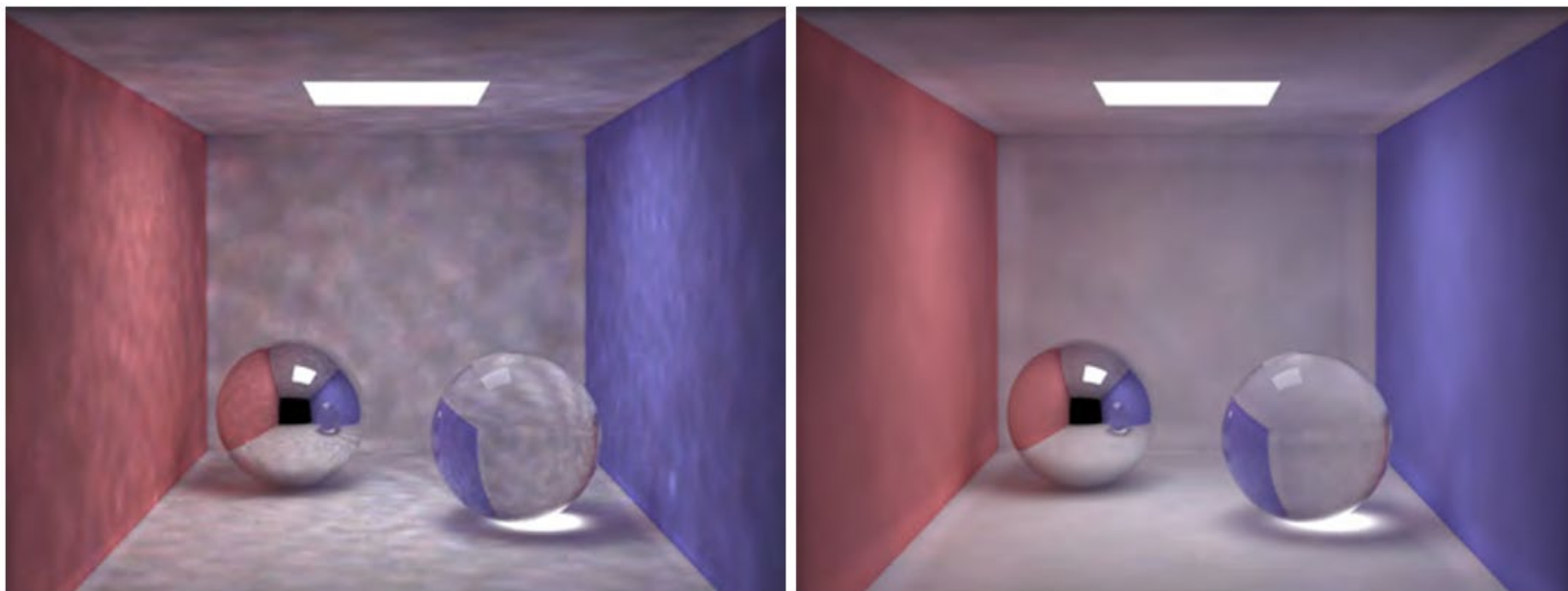
$$L_r(x, \Psi_r) = \int_{\Omega} f_r(x, \Psi_r, \Psi_i) \frac{d^2 \Phi_i(x, \Psi_i)}{dA d\omega_i} d\omega_i$$
$$\approx \sum_{p=1}^N f_r(x, \Psi_r, \Psi_{i,p}) \frac{\Delta \Phi_p(x, \Psi_{i,p})}{\pi r^2}$$

- 参数含义:

- Φ_p : 单个光子 p 的光通量
- $\Psi_{i,p}$: 单个光子 p 的方向
- r : 样本中离 x 最远的样本 x_p 的距离

算法介绍-光子映射 (Photon mapping)

- 光子数量 N 的影响



- 从左至右光子数量分别为100、500

光子映射 vs 路径追踪 算法对比

- 光子映射：从光源出发发射光子；从相机出发进行渲染。
 - 优势：
 - 高效处理焦散等效果。
 - 间接光照更平滑。
 - 光子图一旦建成可以从不同视角快速渲染。
 - 缺点：
 - 内存消耗大。
 - 不擅长处理光滑表面。

光子映射 vs 路径追踪 算法对比

- 路径追踪：从相机出发向场景发射光线。
 - 优势：
 - 内存效率高
 - 通用性强
 - 易于并行化
 - 缺点：
 - 计算焦散效率极低
 - 高方差（噪声）：尤其是在渲染间接光照时，光线路径难以有效到达光源，导致收敛缓慢，图像充满噪点。

实验要求

- 实现漫反射康奈尔盒场景下的光子图建立算法。
- 使用不同数量的光子（10000-100000）对建立好的光子图进行可视化输出。
- 统计并验证场景在不同数量光子下是总能量守恒的。
- 完成“**实验总结**”，篇幅控制在一页左右，以附件的形式上传至小雅平台